Math 502
Homework #2

Name: Gianluca Crescenzo

Exercise 2.2.

(a) Determine the Green's function for the two-point boundary value problem $u''(x) = f(x)$ *on* $0 < x < 1$ *with a Neumann boundary condition at* $x = 0$ *and a Dirichlet condition at* $x = 1$*;i.e., find teh function* $G(x;\bar{x})$ *solving*

$$G''(x;\bar{x}) = \delta(x-\bar{x}), \quad G'(0) = 0, \quad G(1) = 0,$$

the function $G_0(x)$ *solving*

$$G_0''(x) = 0, \quad G_0'(0) = 1, \quad G_0(1) = 0$$

and the function $G_1(x)$ *solving*

$$G_1''(x) = 0, \quad G_1'(0) = 0, \quad G_1(1) = 1.$$

(b) Using this as guidance, find the general formulas for the elements of the inverse of the matrix in equation (2.54). Write out the 5×5 *matrices of* A *and* A^{-1} *for the case* $h = 0.25$*.*

Solution. (a) Since $\delta(x-\bar{x}) = 0$ whenever $x \neq \bar{x}$, $G''(x;\bar{x}) = 0$ whenever $x \neq \bar{x}$. Therefore:

$$G(x;\bar{x}) = \begin{cases} Ax + B, & x \in [0,\bar{x}) \\ Cx + D, & x \in [\bar{x},1] \end{cases}$$

Since Green's function is continuous, it must be the case that $A\bar{x} = B = C\bar{x} = D$. Note that Green's function has a jump condition at $\bar{x}$; i.e.:

$$\int_{\bar{x}-\epsilon}^{\bar{x}+\epsilon} G''(x,\bar{x})dx = \int_{\bar{x}-\epsilon}^{\bar{x}+\epsilon} \delta(x-\bar{x})dx = 1$$

Simplifying the left side gives $G'(\bar{x}+\epsilon;\bar{x}) - G'(\bar{x}-\epsilon;\bar{x}) = 1$; i.e., $C - A = 1$. Now our boundary conditions give that $A = 0$ and $C + D = 0$. From the 4 equations obtained, solving the system yields $A = 0$, $B = \bar{x} - 1$, $C = 1$, and $D = -1$. Thus:

$$G(x;\bar{x}) = \begin{cases} \bar{x} - 1, & x \in [0,\bar{x}) \\ x - 1, & x \in [\bar{x},1] \end{cases}$$

The functions $G_0(x)$ and $G_1(x)$ can be solved by simplifying integrating twice and using the boundary conditions to find the coefficients. We will find that $G_0(x) = x - 1$ and $G_1(x) = 1$.

(b)

$$A = \begin{pmatrix} -4 & 4 & 0 & 0 & 0 \\ 16 & -32 & 16 & 0 & 0 \\ 0 & 16 & -32 & 16 & 0 \\ 0 & 0 & 16 & -32 & 16 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \qquad A^{-1} = \begin{pmatrix} -1 & -\frac{3}{16} & -\frac{1}{8} & -\frac{1}{16} & 1 \\ -\frac{3}{4} & -\frac{3}{16} & -\frac{1}{8} & -\frac{1}{16} & 1 \\ -\frac{1}{2} & -\frac{1}{8} & -\frac{1}{8} & -\frac{1}{16} & 1 \\ -\frac{1}{4} & -\frac{1}{16} & -\frac{1}{16} & -\frac{1}{16} & 1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Exercise 2.3. *(Solvability Condition for Neumann Problem)*
Determine the null space of the matrix A^T, where A is given in Equation (2.58), and verify that the condition (2.62) must hold for the linear system to have solutions.

Solution. Equation (2.58) from the textbook is:

$$\frac{1}{h^2}\begin{pmatrix} -h & h & & & & \\ 1 & -2 & 1 & & & \\ & 1 & -2 & 1 & & \\ & & \ddots & \ddots & \ddots & \\ & & & 1 & -2 & 1 \\ & & & & h & -h \end{pmatrix}\begin{pmatrix} U_0 \\ U_1 \\ U_2 \\ \vdots \\ U_m \\ U_{m+1} \end{pmatrix} = \begin{pmatrix} \sigma_0 + \frac{h}{2}f(x_0) \\ f(x_1) \\ f(x_2) \\ \vdots \\ f(x_m) \\ -\sigma_1 + \frac{h}{2}f(x_{m+1}) \end{pmatrix}$$

The tranpose of the matrix A is:

$$A^T = \frac{1}{h^2}\begin{pmatrix} -h & 1 & & & & \\ h & -2 & 1 & & & \\ & 1 & -2 & \ddots & & \\ & & \ddots & \ddots & 1 & \\ & & & 1 & -2 & h \\ & & & & 1 & -h \end{pmatrix}$$

To find the null space of A^T, we can row reduce the following augmented matrix:

$$\frac{1}{h^2}\left(\begin{array}{cccccc|c} -h & 1 & & & & & 0 \\ h & -2 & 1 & & & & 0 \\ & 1 & -2 & \ddots & & & 0 \\ & & \ddots & \ddots & 1 & & \vdots \\ & & & 1 & -2 & h & 0 \\ & & & & 1 & -h & 0 \end{array}\right).$$

After careful row operations, we get that the null space of A^T is $\text{span}\{(1, h, h, ..., h, 1)^T\}$. Since a singular linear system has a solution if and only if the right-hand side is orthogonal to the null space of A^T, we can conclude that Equation (2.58) has a solution.

Exercise 2.4 prelim. *Find the exact solution to the problem:*

$$u''(x) = e^x, \quad x \in (0,3)$$
$$u(0) = \alpha, \quad u'(3) = \sigma.$$

by simply integrating both sides of the differential equation as many times as necessary. Use this exact solution to test the code in Exercise 2.4. Try a couple of different values of α and σ for yourself, but choose only one set of values to submit as part of your homework solution.

Solution. The solution to this differential equation given the boundary conditions is $u(x) = e^x + (\sigma - e^3)x + (\alpha - 1)$. The values that I will submit for this assignment will be $\sigma = e^3$ and $\alpha = 1$.

Exercise 2.4.

(a) Modify the m-file `bvp_2.m` *so that it implements a Dirichlet boundary condition at $x = a$ and a Neumann condition at $x = b$ and test the modified program.*

(b) Make the same modification to the m-file `bvp_4.m`*, which implements a fourth order accurate method. Again test the modified program.*

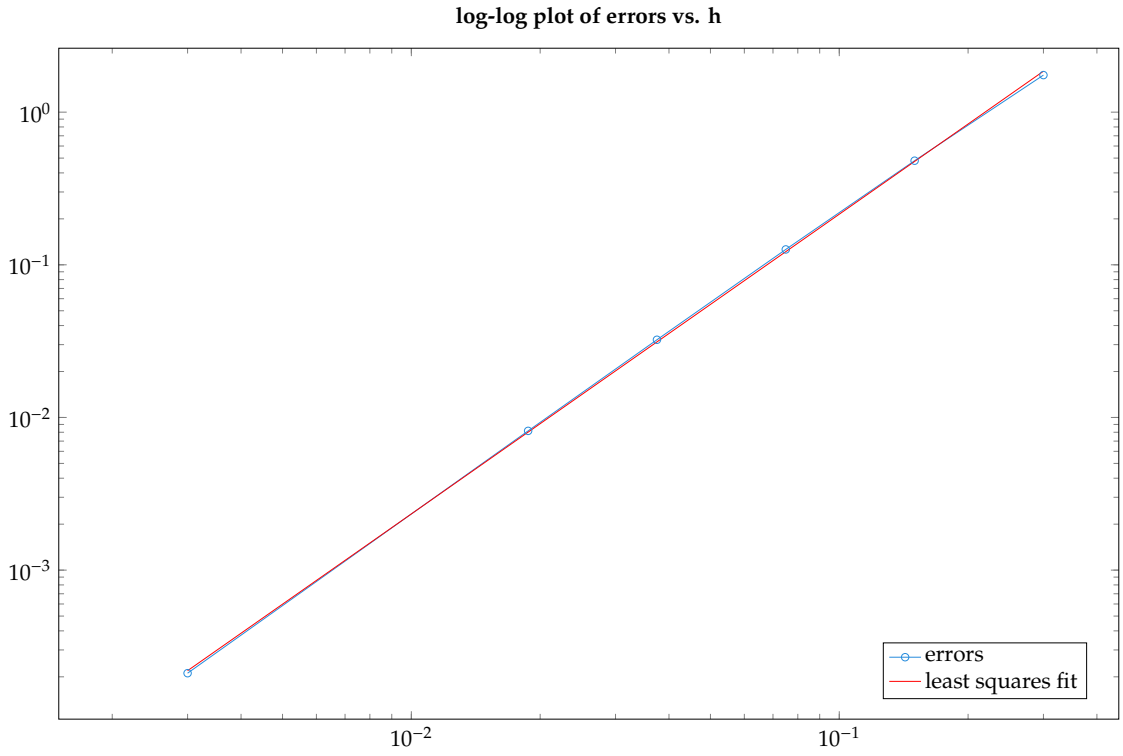
Solution. (a) Here is the adjusted code and a log-log output:

```
% bvp_2.m
% second order finite difference method for the bvp
% u''(x) = f(x), u(ax)=beta, u'(bx)=sigma
% Using 3-pt differences on an arbitrary nonuniform grid.
% Should be 2nd order accurate if grid points vary smoothly, but may
% degenerate to "first order" on random or nonsmooth grids.
% Different BCs can be specified by changing the first and/or last rows of
% A and F.
ax = 0;
bx = 3;
sigma = exp(3); % Neumann boundary condition at bx
beta = 1; % Dirichlet boundary condtion at ax
f = @(x) exp(x); % right hand side function
utrue = @(x) exp(x) + (sigma-exp(bx))*(x - ax) + beta - exp(ax); % true soln
% true solution on fine grid for plotting:
xfine = linspace(ax,bx,101);
ufine = utrue(xfine);
% Solve the problem for ntest different grid sizes to test convergence:
```

```
m1vals = [10 20 40 80 160 1000];
ntest = length(m1vals);
hvals = zeros(ntest,1); % to hold h values
E = zeros(ntest,1); % to hold errors
for jtest=1:ntest
m1 = m1vals(jtest);
m2 = m1 + 1;
m = m1 - 1; % number of interior grid points
hvals(jtest) = (bx-ax)/m1; % average grid spacing, for
   convergence tests
% set grid points:
gridchoice = 'uniform'; % see xgrid.m for other choices
x = xgrid(ax,bx,m,gridchoice);
% set up matrix A (using sparse matrix storage):
A = spalloc(m2,m2,3*m2); % initialize to zero matrix
% first row for Dirichlet BC at ax:
A(1,1:3) = fdcoeffF(0, x(1), x(1:3));
% interior rows:
for i=2:m1
A(i,i-1:i+1) = fdcoeffF(2, x(i), x((i-1):(i+1)));
end
% last row for Neumann BC at bx:
A(m2,m:m2) = fdcoeffF(1, x(m2), x(m:m2));
% Right hand side:
F = f(x);
F(1) = beta;
F(m2) = sigma;
% solve linear system:
U = A\F;
% compute error at grid points:
uhat = utrue(x);
err = U - uhat;
E(jtest) = max(abs(err));
disp(' ')
disp(sprintf('Error with %i points is %9.5e',m2,E(jtest)))
clf
plot(x,U,'o') % plot computed solution
title(sprintf('Computed solution with %i grid points',m2));
hold on
plot(xfine,ufine) % plot true solution
hold off
```

```
% pause to see this plot:
drawnow
input('Hit <return> to continue ');
end
error_table(hvals, E); % print tables of errors and ratios
error_loglog(hvals, E); % produce log
```

```
>> untitled2

Error with 11 points is 1.74886e+00
Hit <return> to continue

Error with 21 points is 4.80811e-01
Hit <return> to continue

Error with 41 points is 1.26086e-01
Hit <return> to continue

Error with 81 points is 3.22858e-02
Hit <return> to continue

Error with 161 points is 8.16884e-03
Hit <return> to continue

Error with 1001 points is 2.11242e-04
Hit <return> to continue

      h         error        ratio       observed order
   0.30000   1.74886e+00        NaN              NaN
   0.15000   4.80811e-01    3.63732          1.86288
   0.07500   1.26086e-01    3.81336          1.93106
   0.03750   3.22858e-02    3.90531          1.96544
   0.01875   8.16884e-03    3.95231          1.98270
   0.00300   2.11242e-04   38.67058          1.99450
```



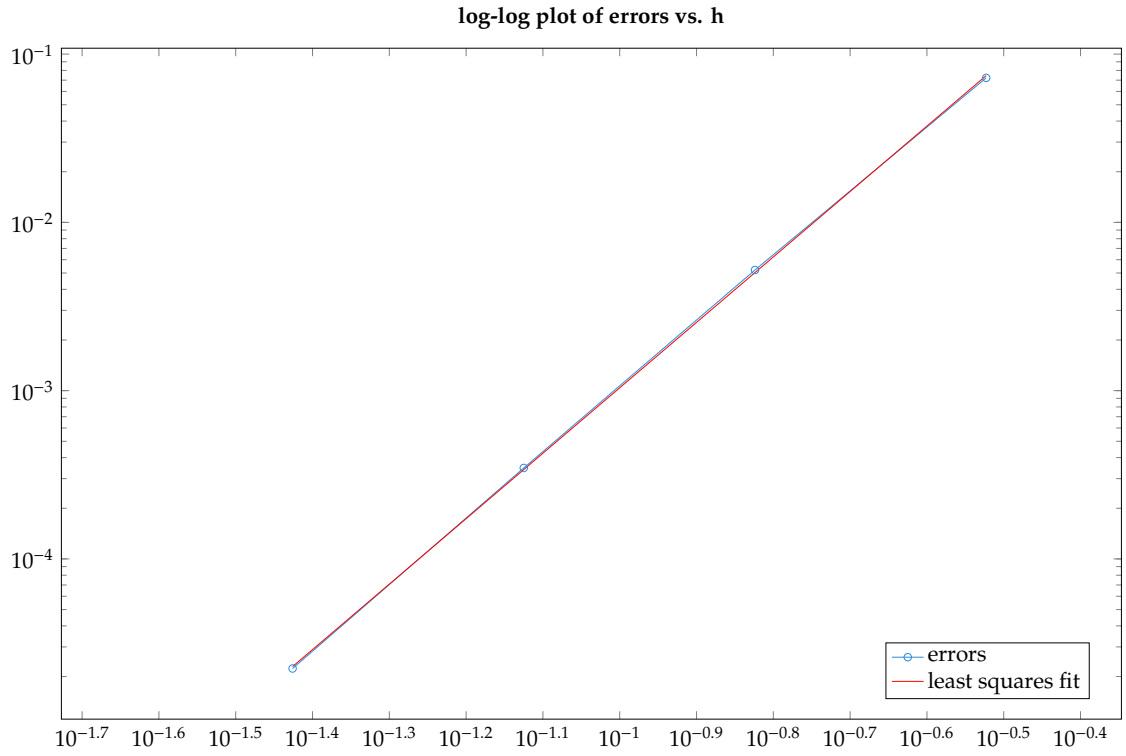
(b) Here is the adjusted code and a log-log output:

```
% bvp4.m
% second order finite difference method for the bvp
% u''(x) = f(x), u(ax)=beta, u'(bx)=sigma
% fourth order finite difference method for the bvp
% u'' = f, u(ax)=beta, u'(bx)=sigma
% Using 5-pt differences on an arbitrary grid.
% Should be 4th order accurate if grid points vary smoothly.
% Different BCs can be specified by changing the first and/or
    last rows of
% A and F.
ax = 0;
bx = 3;
sigma = exp(3); % Neumann boundary condition at bx
beta = 1; % Dirichlet boundary condtion at ax
f = @(x) exp(x); % right hand side function
utrue = @(x) exp(x) + (sigma-exp(bx))*(x - ax) + beta - exp(
   ax); % true soln
% true solution on fine grid for plotting:
xfine = linspace(ax,bx,101);
ufine = utrue(xfine);
```

```
% Solve the problem for ntest different grid sizes to test
   convergence:
m1vals = [10 20 40 80];
ntest = length(m1vals);
hvals = zeros(ntest,1); % to hold h values
E = zeros(ntest,1); % to hold errors
for jtest=1:ntest
m1 = m1vals(jtest);
m2 = m1 + 1;
m = m1 - 1; % number of interior grid points
hvals(jtest) = (bx-ax)/m1; % average grid spacing, for
   convergence tests
% set grid points:
gridchoice = 'uniform';
x = xgrid(ax,bx,m,gridchoice);
% set up matrix A (using sparse matrix storage):
A = spalloc(m2,m2,5*m2); % initialize to zero matrix
% first row for Dirichlet BC on u(x(1))
A(1,1:5) = fdcoeffF(0, x(1), x(1:5));
% second row for u''(x(2))
A(2,1:6) = fdcoeffF(2, x(2), x(1:6));
% interior rows:
for i=3:m
A(i,i-2:i+2) = fdcoeffF(2, x(i), x((i-2):(i+2)));
end
% next to last row for u''(x(m+1))
A(m1,m-3:m2) = fdcoeffF(2,x(m1),x(m-3:m2));
% last row for Neumann BC on u'(x(m+2))
A(m2,m-2:m2) = fdcoeffF(1,x(m2),x(m-2:m2));
% Right hand side:
F = f(x);
F(1) = beta;
F(m2) = sigma;
% solve linear system:
U = A\F;
% compute error at grid points:
uhat = utrue(x);
err = U - uhat;
E(jtest) = max(abs(err));
disp(' ')
disp(sprintf('Error with %i points is %9.5e',m2,E(jtest)))
```

```
clf
plot(x,U,'o') % plot computed solution
title(sprintf('Computed solution with %i grid points',m2));
hold on
plot(xfine,ufine) % plot true solution
hold off
% pause to see this plot:
drawnow
input('Hit <return> for next plot ');
end
error_table(hvals, E); % print tables of errors and ratios
error_loglog(hvals, E); % produce log
```

```
>> untitled3

Error with 11 points is 7.22610e-02
Hit <return> for next plot

Error with 21 points is 5.21206e-03
Hit <return> for next plot

Error with 41 points is 3.47061e-04
Hit <return> for next plot

Error with 81 points is 2.23260e-05
Hit <return> for next plot

      h         error        ratio       observed order
   0.30000   7.22610e-02        NaN              NaN
   0.15000   5.21206e-03   13.86421          3.79329
   0.07500   3.47061e-04   15.01768          3.90859
   0.03750   2.23260e-05   15.54515          3.95839


Least squares fit gives E(h) = 8.04933 * h^3.88894
```



Exercise 2.6. *Consider the following linear boundary value problem with Dirichlet boundary conditions:*

$$u''(x) = u(x) = 0, \quad x \in (a, b)$$
$$u(a) = \alpha, \quad u(b) = \beta.$$

Note that this equation arises from a linearized pendulum.

(a) Modify the m-file `bvp_2.m` *to solve this problem. Test your modified routine on the problem with* $a = 0$, $b = 1$, $\alpha = 2$, $\beta = 3$. *Determine the exact solution for comparison.*

(b) Let $a = 0$ *and* $b = \pi$. *For what values of* α *and* β *does this boundary value problem have solutions? Sketch a family of solutions in a case where there are infinitely many solutions*

(c) Solve the problem with $a = 0$, $b = \pi$, $\beta = -1$ *using your modified* `bvp_2.m`. *Which solution to the boundary value problem does this appear to converge to as* $h \to 0$? *Change the boundary value at* $b = \pi$ *to* $\beta = 1$. *Now how does the numerical solution behave as* $h \to 0$.

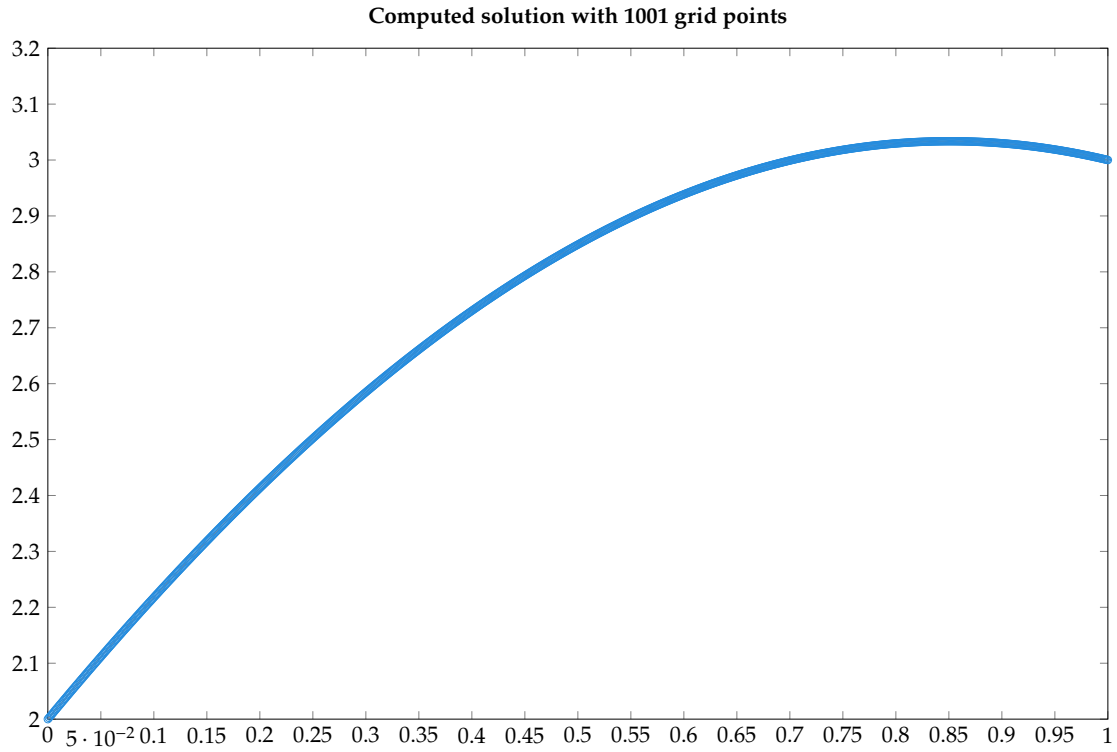
(d) You might expect the linear system in (c) to be singular since the boundary value problem is not well posed. It is not, because of discretization error. Compute the eigenvalues of the matrix A *for this problem and show that an eigenvalue approaches 0 as* $h \to 0$. *Also show that* $\left\|A^{-1}\right\|_2$ *blows up as* $h \to 0$ *so that the discretization is unstable.*

Solution. (a) Here is the modified code and output:

```
% bvp_2.m
% second order finite difference method for the bvp
% u''(x) + u(x) = 0, u(ax)=alpha, u(bx)=beta
% Using 3-pt differences on an arbitrary nonuniform grid.
% Should be 2nd order accurate if grid points vary smoothly, but may
% degenerate to "first order" on random or nonsmooth grids.
%
% Different BCs can be specified by changing the first and/or last rows of
% A and F.
%
% From http://www.amath.washington.edu/~rjl/fdmbook/ (2007)
ax = 0;
bx = 1;
alpha = 2; % Dirichlet boundary condition at ax
beta = 3; % Dirichlet boundary condtion at bx
f = @(x) 0*x; % right hand side function
utrue = @(x) alpha*cos(x) + (beta-alpha*cos(bx))*sin(x)/sin(bx); % true soln
% true solution on fine grid for plotting:
xfine = linspace(ax,bx,101);
ufine = utrue(xfine);
% Solve the problem for ntest different grid sizes to test convergence:
m1vals = [10 20 40 80 160 1000];
ntest = length(m1vals);
hvals = zeros(ntest,1); % to hold h values
E = zeros(ntest,1); % to hold errors
for jtest=1:ntest
m1 = m1vals(jtest);
m2 = m1 + 1;
m = m1 - 1; % number of interior grid points
hvals(jtest) = (bx-ax)/m1; % average grid spacing, for convergence tests
% set grid points:
gridchoice = 'uniform'; % see xgrid.m for other choices
x = xgrid(ax,bx,m,gridchoice);
% set up matrix A (using sparse matrix storage):
A = spalloc(m2,m2,3*m2); % initialize to zero matrix
```

```
% first row for Dirichlet BC at ax:
A(1,1:3) = fdcoeffF(0, x(1), x(1:3));
% interior rows:
for i=2:m1
A(i,i-1:i+1) = fdcoeffF(2, x(i), x((i-1):(i+1)));
A(i,i) = A(i,i) + 1;
end
% last row for Dirichlet BC at bx:
A(m2,m:m2) = fdcoeffF(0,x(m2),x(m:m2));
% Right hand side:
F = f(x);
F(1) = alpha;
F(m2) = beta;
% solve linear system:
U = A\F;
% compute error at grid points:
uhat = utrue(x);
err = U - uhat;
E(jtest) = max(abs(err));
disp(' ')
disp(sprintf('Error with %i points is %9.5e',m2,E(jtest)))
clf
plot(x,U,'o') % plot computed solution
title(sprintf('Computed solution with %i grid points',m2));
hold on
plot(xfine,ufine) % plot true solution
hold off
% pause to see this plot:
drawnow
input('Hit <return> to continue ');
end
error_table(hvals, E); % print tables of errors and ratios
error_loglog(hvals, E); % produce log
```

```
>> problem2point6

Error with 11 points is 3.24687e-04
Hit <return> to continue

Error with 21 points is 8.10848e-05
Hit <return> to continue

Error with 41 points is 2.02705e-05
Hit <return> to continue

Error with 81 points is 5.06999e-06
Hit <return> to continue

Error with 161 points is 1.26748e-06
Hit <return> to continue

Error with 1001 points is 3.24062e-08
Hit <return> to continue

      h        error       ratio       observed order
   0.10000   3.24687e-04       NaN             NaN
   0.05000   8.10848e-05   4.00430         2.00155
   0.02500   2.02705e-05   4.00014         2.00005
   0.01250   5.06999e-06   3.99812         1.99932
   0.00625   1.26748e-06   4.00007         2.00002
   0.00100   3.24062e-08  39.11215         2.00069



Least squares fit gives E(h) = 0.0324794 * h^2.00029

>>
```



The exact solution for $a = 0$, $b = 1$, $\alpha = 2$, and $\beta = 3$ is $u(x) = 2\cos(x) + \frac{3-2\cos(1)}{\sin(1)}\sin(x)$.

(b) The boundary value problem has a solution if and only if $\beta = -\alpha$. There are infinitely many solutions

(c) Observe the following code, output, and plot.

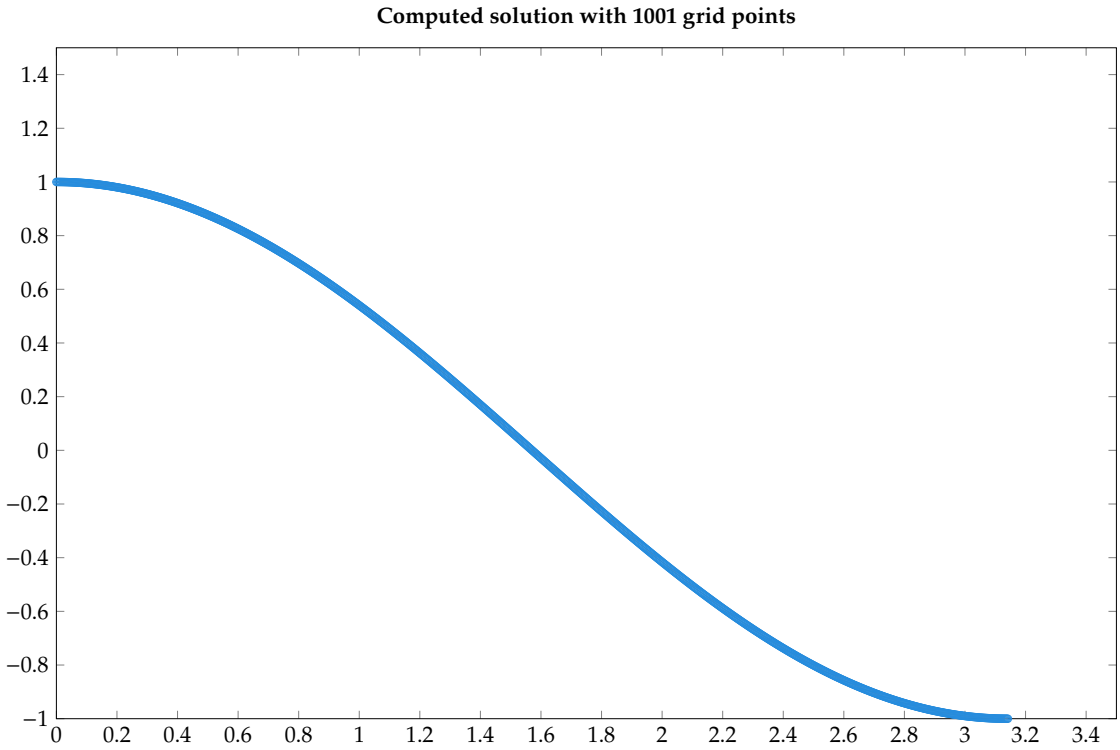
```
% bvp_2.m
% second order finite difference method for the bvp
% u''(x) + u(x) = 0, u(ax)=alpha, u(bx)=beta
% Using 3-pt differences on an arbitrary nonuniform grid.
% Should be 2nd order accurate if grid points vary smoothly, but may
% degenerate to "first order" on random or nonsmooth grids.
%
% Different BCs can be specified by changing the first and/or last rows of
% A and F.
%
% From http://www.amath.washington.edu/~rjl/fdmbook/ (2007)
ax = 0;
bx = pi;
alpha = 1;
beta = -1; % first run
f = @(x) 0*x; % right hand side function
```

```
utrue = @(x) cos(x); % for comparison to the converged
   solution % true soln
% true solution on fine grid for plotting:
xfine = linspace(ax,bx,101);
ufine = utrue(xfine);
% Solve the problem for ntest different grid sizes to test
   convergence:
m1vals = [10 20 40 80 160 1000];
ntest = length(m1vals);
hvals = zeros(ntest,1); % to hold h values
E = zeros(ntest,1); % to hold errors
for jtest=1:ntest
m1 = m1vals(jtest);
m2 = m1 + 1;
m = m1 - 1; % number of interior grid points
hvals(jtest) = (bx-ax)/m1; % average grid spacing, for
   convergence tests
% set grid points:
gridchoice = 'uniform'; % see xgrid.m for other choices
x = xgrid(ax,bx,m,gridchoice);
% set up matrix A (using sparse matrix storage):
A = spalloc(m2,m2,3*m2); % initialize to zero matrix
% first row for Dirichlet BC at ax:
A(1,1:3) = fdcoeffF(0, x(1), x(1:3));
% interior rows:
for i=2:m1
A(i,i-1:i+1) = fdcoeffF(2, x(i), x((i-1):(i+1)));
A(i,i) = A(i,i) + 1;
end
% last row for Dirichlet BC at bx:
A(m2,m:m2) = fdcoeffF(0,x(m2),x(m:m2));
% Right hand side:
F = f(x);
F(1) = alpha;
F(m2) = beta;
% solve linear system:
U = A\F;
% compute error at grid points:
uhat = utrue(x);
err = U - uhat;
E(jtest) = max(abs(err));
```

```
disp(' ')
disp(sprintf('Error with %i points is %9.5e',m2,E(jtest)))
clf
plot(x,U,'o') % plot computed solution
title(sprintf('Computed solution with %i grid points',m2));
hold on
plot(xfine,ufine) % plot true solution
hold off
% pause to see this plot:
drawnow
input('Hit <return> to continue ');
end
error_table(hvals, E); % print tables of errors and ratios
matlab2tikz('myfile2.tex');
error_loglog(hvals, E); % produce log
```

```
>> problem2point6

Error with 11 points is 2.31489e-03
Hit <return> to continue

Error with 21 points is 5.73244e-04
Hit <return> to continue

Error with 41 points is 1.44354e-04
Hit <return> to continue

Error with 81 points is 3.60616e-05
Hit <return> to continue

Error with 161 points is 9.01416e-06
Hit <return> to continue

Error with 1001 points is 7.74535e-06
Hit <return> to continue

      h         error        ratio       observed order
   0.31416   2.31489e-03        NaN              NaN
   0.15708   5.73244e-04    4.03823          2.01372
   0.07854   1.44354e-04    3.97111          1.98954
   0.03927   3.60616e-05    4.00297          2.00107
   0.01963   9.01416e-06    4.00055          2.00020
```

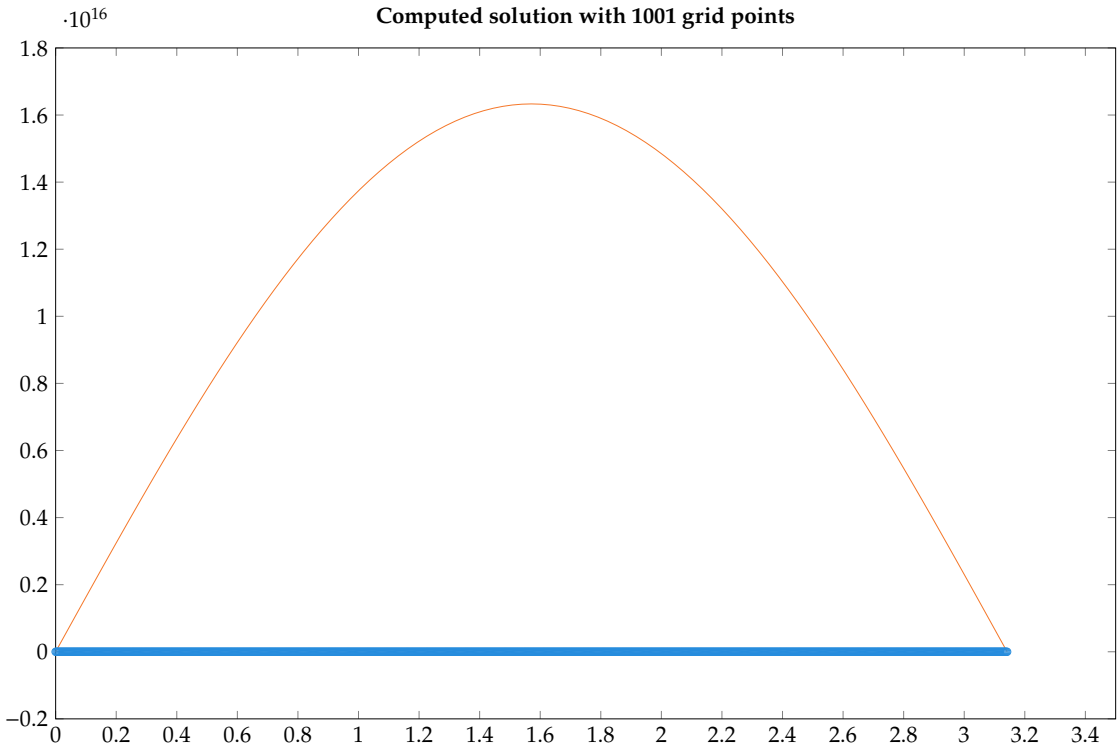
```
   0.00314    7.74535e-06    1.16382          0.08278

Least squares fit gives E(h) = 0.00483763 * h^1.29861
```



We can see that this solution converges to $u(x) = \cos(x)$. Making the minimal change of $\beta = 1$ gives the following output and plot:

```
>> problem2point6

Error with 11 points is 1.63312e+16
Hit <return> to continue

Error with 21 points is 1.63312e+16
Hit <return> to continue

Error with 41 points is 1.63312e+16
Hit <return> to continue

Error with 81 points is 1.63312e+16
Hit <return> to continue
```

```
Error with 161 points is 1.63312e+16
Hit <return> to continue

Error with 1001 points is 1.63312e+16
Hit <return> to continue

      h         error        ratio       observed order
   0.31416   1.63312e+16        NaN              NaN
   0.15708   1.63312e+16    1.00000         -0.00000
   0.07854   1.63312e+16    1.00000         -0.00000
   0.03927   1.63312e+16    1.00000         -0.00000
   0.01963   1.63312e+16    1.00000         -0.00000
   0.00314   1.63312e+16    1.00000         -0.00000

Least squares fit gives E(h) = 1.63312e+16 * h^-1.9068e-11

>>
```



Since solvability requires $\beta = -\alpha$, the discretized solution does not converge to the true solution.

(d)